

Assignment #3 — Problem 1,2

G202558003 김지혜

1 문제 1 : Exploration of the Marabou Resources Directory

The `resources/` directory of the Marabou repository contains benchmark models, property files, and example scripts used to reproduce experimental results from the original tool paper.

Models. All provided models are stored in the `resources/nnet/` folder in the `.nnet` format and are organized into three benchmark categories: (1) **ACAS Xu** — 45 fully-connected networks (5 inputs, 5 outputs) for airborne collision avoidance, originally from the Reluplex paper; (2) **coav** — CollisionAvoidance networks from the Planet paper; and (3) **twin** — TwinStreams networks from the Planet repository. Marabou 2.0 additionally supports `.onnx` (primary) and `.pb` (TensorFlow) formats, as well as VNNLIB property files.

Verification properties. The `resources/properties/` folder contains plain-text property files (e.g. `acas_property_3.txt`) that define input bounds and output constraints for ACAS Xu safety verification.

Example scripts and notebooks. `resources/runMarabou.py` provides a command-line interface supporting MNIST and CIFAR-10 ℓ_∞ robustness queries in addition to ACAS Xu benchmarks. `resources/SplitAndConquerGuide.ipynb` is a Jupyter notebook demonstrating Marabou’s Split-and-Conquer (S&C) parallel verification algorithm with parameter tuning guidance.

2 문제 2 : Model, Dataset, and Verification Query

Model. We trained a small fully-connected network on MNIST using PyTorch. The architecture is:

$$\text{Input}(784) \rightarrow \text{Linear}(32) \rightarrow \text{ReLU} \rightarrow \text{Linear}(16) \rightarrow \text{ReLU} \rightarrow \text{Linear}(10)$$

The model was exported to ONNX format (`models/mnist_fc.onnx`, opset 11) for use with Marabou’s `maraboupy` interface. This model is not present in Marabou’s resource directory, making it a valid external benchmark. After 5 epochs of Adam optimization, the model achieved **95.93% test accuracy** on the MNIST test set.

Dataset. The MNIST handwritten digit dataset (60,000 training / 10,000 test images, 28×28 grayscale, pixel values in $[0, 1]$) was used. Verification was performed on the **first test sample**, a digit **7**.

Verification query. We verified *local robustness*: for the selected test image $\mathbf{x}$ classified as digit 7, we checked that every input $\mathbf{x}'$ satisfying $\|\mathbf{x}' - \mathbf{x}\|_\infty \leq \varepsilon$ is also classified as 7. Formally, for each class $j \neq 7$, Marabou searched for $\mathbf{x}'$ in the ℓ_∞ -ball such that $f_j(\mathbf{x}') \geq f_7(\mathbf{x}')$ (i.e., class j beats the true label). This was encoded as:

$$-f_j(\mathbf{x}') + f_7(\mathbf{x}') \leq 0$$

with input bounds $x_i - \varepsilon \leq x'_i \leq x_i + \varepsilon$ (clipped to $[0, 1]$) for each pixel i . We used $\varepsilon = 0.01$.

3 Results and Interpretation

Table 1 summarizes the verification results for all 9 class pairs.

Query	Result	Time (s)
class 7 vs. 0	UNSAT	0.0
class 7 vs. 1	UNSAT	0.0
class 7 vs. 2	UNSAT	0.0
class 7 vs. 3	UNSAT	0.1
class 7 vs. 4	UNSAT	0.0
class 7 vs. 5	UNSAT	0.1
class 7 vs. 6	UNSAT	0.0
class 7 vs. 8	UNSAT	0.0
class 7 vs. 9	UNSAT	0.0

Table 1: Verification results for digit 7, $\varepsilon = 0.01$.

All 9 queries returned **UNSAT**, meaning no adversarial example exists within the ℓ_∞ -ball of radius 0.01 around the test image. The total verification time was approximately **0.3 seconds**. This confirms that the model is locally robust for this input: perturbations of up to 1% in pixel intensity cannot change the predicted digit.

One notable observation arose during implementation: an incorrect sign in the output constraint ($[1, -1]$ instead of $[-1, 1]$) caused Marabou to check the wrong direction (whether the true label beats class j , which is trivially SAT), yielding a spurious SAT result in 0.1 s. After correcting the constraint direction, all queries correctly returned UNSAT. This highlighted the importance of carefully encoding the negation of the desired property when using Marabou’s inequality API.

4 Discussion: Strengths and Limitations of Marabou

Strengths. Marabou provides *complete* verification guarantees: an UNSAT result is a formal proof that the property holds for all inputs in the specified region, not merely a statistical estimate. The Python API (`maraboupy`) is intuitive and supports ONNX models directly, making it easy to integrate with standard deep learning workflows. Installation via `pip install maraboupy` is straightforward. The Split-and-Conquer mode enables parallelism, which can significantly reduce wall-clock time on multi-core servers.

Limitations. Marabou scales poorly with model size. Even for this small network (784–32–16–10), verification time grows rapidly as ε increases due to the exponential search space of the underlying SMT solver. Larger models (e.g., full ResNets) are intractable. Additionally, the ONNX parser requires `onnxruntime` as a separate dependency, which is not automatically installed with `maraboupy` and caused an import error during setup. The inequality API requires careful attention to constraint direction, as incorrect encoding can produce misleading SAT results without any explicit warning.